MOBL-12: Advanced Instrumentation & Optimization

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 12 of 12 | **Created:** February 2026 | **Last Updated:** 11/11/2026

Overview

This notebook covers advanced mobile SDK techniques that go beyond automatic instrumentation. You will learn how to send custom business events, create manual and nested user actions, report errors and events programmatically, tag web requests for end-to-end tracing, instrument A/B tests and feature flags, tune SDK performance, and manage multi-app monitoring strategies. These techniques give you fine-grained control over what mobile telemetry reaches Grail and how it maps to your business logic.

Table of Contents

1. Custom Business Events
2. Advanced User Actions
3. Error & Event Reporting APIs
4. Web Request Tagging
5. A/B Testing & Feature Flags
6. SDK Performance Optimization
7. Multi-App Strategies

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:bizevents:read` (custom business events), `storage:user.events:read`, `storage:user.sessions:read`
Mobile App	iOS or Android app with Dynatrace SDK integrated
Prior Knowledge	Familiarity with MOBL-01 through MOBL-09 (fundamentals, SDK setup, user actions, crash reporting, network monitoring)
SDK Version	Dynatrace iOS Agent 8.x+ or Android Agent 8.x+

1. Custom Business Events

The `sendBizEvent()` API allows you to send custom business events directly from your mobile app into Grail. Unlike auto-captured user actions, business events let you track domain-specific interactions that matter to your business -- purchases, sign-ups, feature usage, subscription changes, and more.

Why Custom Business Events?

Use Case	Example	Business Value
Purchases	In-app purchase completed	Revenue tracking, conversion funnels
Sign-ups	New account registration	Growth metrics, onboarding analysis
Feature usage	User opened a specific feature	Feature adoption, prioritization
Content engagement	Article read, video watched	Content strategy optimization
Error context	User-reported issue with metadata	Correlate user feedback with technical data

iOS Implementation

```
// iOS -- send a purchase business event
let attributes: [String: Any] = [
    "product.name": "Premium Subscription",
    "product.price": 9.99,
    "currency": "USD",
    "payment.method": "apple_pay"
]
Dynatrace.sendBizEvent(withType: "com.myapp.purchase", attributes: attributes)
```

Android Implementation

```
// Android -- send a purchase business event
// sendBizEvent(String type, JSONObject attributes) takes a JSONObject, not a
Map
JSONObject().apply {
    put("product.name", "Premium Subscription")
    put("product.price", 9.99)
    put("currency", "USD")
    put("payment.method", "google_pay")
}.also { jsonObject -&gt;
    Dynatrace.sendBizEvent("com.myapp.purchase", jsonObject)
}
```

Best Practices for Business Events

- **Use reverse-domain naming** for `event.type` (e.g., `com.myapp.purchase`) to avoid collisions
- **Keep attribute names consistent** across iOS and Android implementations
- **Include version context** -- add `app.version` as an attribute if not automatically enriched
- **Avoid PII** -- never include email addresses, phone numbers, or other personal data in business event attributes
- **Only monitored sessions send them** -- business events are captured only for monitored sessions; when OneAgent is disabled by a flag or by cost and traffic control, they are not reported (OneAgent for iOS/Android 8.253+)

> **Sources:** OneAgent SDK for iOS (DT docs), OneAgent SDK for Android (DT docs) — *"With sendBizEvent , you can report business events. These are standalone events, as Dynatrace sends them separately from user actions or user sessions."*

Query Custom Purchase Events

Use the following DQL query to retrieve and inspect custom purchase business events sent from your mobile app:

```
// Query custom purchase business events
fetch bizevents, from:-24h
| filter event.type == "com.myapp.purchase"
| fields timestamp, product.name, product.price, currency, payment.method
| sort timestamp desc
| limit 50
```

2. Advanced User Actions

While the Dynatrace SDK auto-instruments many user actions (taps, screen loads), advanced scenarios require manual control. This section covers nested actions, long-running actions, and action properties.

Nested Actions

Nested actions let you create parent-child relationships between user actions. This is useful for multi-step flows where you want to measure both the overall flow and individual steps.

```
// iOS -- nested actions for a checkout flow
let parentAction = DTXAction.enter(withName: "Checkout Flow")
let childAction = DTXAction.enter(withName: "Validate Cart", parentAction:
parentAction)
// ... validation logic ...
childAction?.leave()
// ... more checkout logic ...
parentAction?.leave()
```

```
// Android -- nested actions for a checkout flow
val parentAction = Dynatrace.enterAction("Checkout Flow")
val childAction = Dynatrace.enterAction("Validate Cart", parentAction)
// ... validation logic ...
childAction.leaveAction()
// ... more checkout logic ...
parentAction.leaveAction()
```

Long-Running Actions

Auto-generated actions close after a short inactivity timeout set by the agent. For long-running operations like file uploads, background syncs, or multi-screen wizards, you need to manage the action lifecycle explicitly:

```
// iOS -- long-running action for file upload
let uploadAction = DTXAction.enter(withName: "Upload Photo")
uploadPhoto { result in
    switch result {
    case .success:
        uploadAction?.reportValue(withName: "upload.size_bytes", intValue:
Int64(fileSize))
        uploadAction?.leave()
    case .failure(let error):
        uploadAction?.reportError(withName: "upload.failed", error: error)
        uploadAction?.leave()
    }
}
```

Action Properties

Attach custom properties to user actions for richer analysis:

Method	Type	Example
`reportValue(withName:intValue:)`	Integer	`action.reportValue(withName: "items_in_cart", intValue: 3)`
`reportValue(withName:doubleValue:)`	Double	`action.reportValue(withName: "total_price", doubleValue: 49.99)`

Method	Type	Example
`reportValue(withName:stringValue:)`	String	`action.reportValue(withName: "category", stringValue: "electronics")`
`reportError(withName:error:)`	Error	`action.reportError(withName: "validation_error", error: err)`

> **Tip:** Action properties become queryable fields in Grail. Use consistent naming across platforms so your DQL queries work for both iOS and Android data.

3. Error & Event Reporting APIs

Beyond automatic crash detection, the SDK provides APIs to report handled errors, custom error conditions, and diagnostic events. This is critical for capturing errors that your app handles gracefully (try/catch) but that you still want visibility into.

Reporting Handled Errors

```
// iOS -- report a handled error
do {
    let data = try parseUserProfile(json)
} catch {
    DTXAction.reportError(withName: "profile_parse_error", error: error) //
standalone error
}
```

```
// Android -- report a handled error
try {
    val data = parseUserProfile(json)
} catch (e: Exception) {
    Dynatrace.reportError("profile_parse_error", e)
}
```

Reporting Custom Events

`reportEvent` is an **action** method and takes only a name -- the event must belong to an open action. For an event with attributes, send a business event instead:

```
// iOS -- a named event inside an action
let storageCheck = DTXAction.enter(withName: "Check storage")
storageCheck?.reportEvent(withName: "low_storage_warning")
storageCheck?.leave()

// iOS -- the same signal with attributes, as a standalone business event
Dynatrace.sendBizEvent(withType: "com.myapp.low_storage_warning", attributes:
[
    "available_mb": availableMB,
    "threshold_mb": 100
])
```

> **Correction (09/28/2026).** Earlier revisions called `Dynatrace.reportError(withName:error:)` and `Dynatrace.reportEvent(withName:attributes:)` on iOS. Neither exists: errors are reported with `DTXAction.reportError(withName:error:)` (standalone) or on an action, and events with `action.reportEvent(withName:)`.
>
> **Sources:** OneAgent SDK for iOS (DT docs) — *"The event must belong to an existing custom action or an autogenerated user action ."*

Query Reported Errors

Use the following query to retrieve errors that were explicitly reported from your mobile SDK:

```
// Reported errors from mobile apps (API-reported errors)
fetch user.events, from:-24h
| filter in(dt.rum.agent.type, {"android", "ios"})
| filter characteristics.has_error and characteristics.is_api_reported
| fields start_time, frontend.name, error.name, error.code, os.name,
app.short_version
| sort start_time desc
| limit 50
```

Error Reporting Best Practices

- **Report all handled exceptions** that indicate degraded functionality -- even if the user does not see an error message
- **Include context** -- attach relevant metadata (user action, screen name, request URL) to help with root cause analysis
- **Avoid high-volume reporting** -- do not report transient conditions (e.g., network retries) that self-resolve. Focus on errors that impact the user experience
- **Use consistent error names** across platforms so a single DQL query can aggregate iOS and Android errors together

4. Web Request Tagging

Web request tagging links mobile-initiated HTTP requests to their corresponding server-side traces. This enables true end-to-end distributed tracing from the user's device through your backend services.

How It Works

1. The mobile SDK generates a unique request tag for each outgoing HTTP request
2. Your app adds this tag as a custom HTTP header (`x-dynatrace`)
3. The Dynatrace server-side agent (OneAgent) reads the header and correlates the server-side span with the mobile user action
4. The result is a single distributed trace spanning device to backend

iOS Implementation

```
// iOS -- tag an outgoing web request with a parent action
let url = URL(string: "https://api.myapp.com/checkout")!
var request = URLRequest(url: url)
let checkout = DTXAction.enter(withName: "Checkout")
if let dynatraceHeaderValue = checkout?.getTagFor(url) {
    let dynatraceHeaderKey = Dynatrace.getRequestTagHeader() // always "x-
dynatrace"
    request.setValue(dynatraceHeaderValue, forHTTPHeaderField:
dynatraceHeaderKey)
}
// Without a parent action: Dynatrace.getRequestTagValue(for: url)
let task = URLSession.shared.dataTask(with: request) { data, response, error
in
    // handle response
    checkout?.leave()
}
task.resume()
```

Android Implementation

```
// Android -- tag an outgoing web request with a parent action
val webAction = Dynatrace.enterAction("Checkout")
val uniqueRequestTag = webAction.getRequestTag()          // standalone:
Dynatrace.getRequestTag()
val timing = Dynatrace.getWebRequestTiming(uniqueRequestTag)
val request = Request.Builder()
    .url("https://api.myapp.com/checkout")
    .addHeader(Dynatrace.getRequestTagHeader(), uniqueRequestTag)
    .build()
timing.startWebRequestTiming()
// ... execute the request, then stop the timing with the URL and response
code ...
webAction.leaveAction()
```

> **Correction (09/28/2026).** Earlier revisions used `DTXAction.getRequestTag(for:)` (iOS) and `Dynatrace.getRequestTag(connection)` (Android). The documented calls are `action?.getTagFor(url)` / `Dynatrace.getRequestTagValue(for:)` on iOS and `action.getRequestTag()` / `Dynatrace.getRequestTag()` on Android, with the header name from `Dynatrace.getRequestTagHeader()`.

> **Sources:** OneAgent SDK for iOS (DT docs), OneAgent SDK for Android (DT docs) — *"To track web requests, add the x-dynatrace HTTP header with a unique value to the web request."*

When to Use Manual Tagging

Scenario	Auto-tagged?	Manual Tagging Needed?
URLSession (iOS), and libraries built on it such as Alamofire	Yes	No
HttpURLConnection and OkHttp (Android), and libraries built on them	Yes	No
Other HTTP frameworks	No	Yes
WebSocket connections (`ws://` , `wss://`)	No	Yes
gRPC calls	No	Yes
GraphQL over custom transport	No	Yes

> **Note:** Most standard HTTP libraries are auto-instrumented by the SDK. Manual tagging is primarily needed for custom or non-standard network transports.
>
> **Sources:** Support and limitations — Android (DT docs) — *"Only web requests from the frameworks HttpURLConnection and OkHttp (including frameworks that are based on these frameworks) are automatically instrumented"*; *"webSocket requests (ws://, wss://) and other non-HTTP protocols require manual instrumentation."*

5. A/B Testing & Feature Flags

Instrumenting A/B tests and feature flags in Dynatrace lets you correlate experiment variants with real user performance and business outcomes. By reporting variant assignments as session properties and business events, you can answer questions like: "Does variant B of the checkout flow have a higher crash rate?" or "Which feature flag configuration drives more conversions?"

Reporting A/B Test Variants

Use session properties to tag the user's session with the assigned variant:

Values are reported on a user action, then converted into a session property in the app settings:

```
// iOS -- report the A/B test variant on an action
let assign = DTXAction.enter(withName: "Assign experiment")
assign?.reportValue(withName: "ab_test_checkout_v2", stringValue: "variant_b")
assign?.leave()
```

```
// Android -- report the A/B test variant on an action
val assign = Dynatrace.enterAction("Assign experiment")
assign.reportValue("ab_test_checkout_v2", "variant_b")
assign.leaveAction()
```

Reporting Feature Flag States

Send business events when feature flags are evaluated so you can track which users see which features:

```
// iOS -- report feature flag evaluation as business event
let flagAttributes: [String: Any] = [
    "feature.name": "dark_mode",
    "feature.variant": "enabled",
    "feature.source": "launchdarkly"
]
Dynatrace.sendBizEvent(withType: "com.myapp.feature_flag", attributes:
flagAttributes)
```

Track App Version Adoption Over Time

App version adoption is closely related to feature flag rollouts. Use this query to see how session volume (`user.sessions` , one record per session) distributes across app versions (`app.short_version`):

```
// Track app version adoption over time
fetch user.sessions, from:-7d
| filter dt.rum.application.type == "mobile"
| filter isNotNull(app.short_version)
| makeTimeseries session_count = count(), by:{app.short_version},
time:start_time, interval:24h
```

6. SDK Performance Optimization

The Dynatrace mobile SDK is designed to be lightweight, but in performance-sensitive applications you may need to fine-tune its behavior. This section covers the key optimization levers.

Optimization Levers

Optimization	Description	Impact
Beacon batching	SDK batches beacons before sending	Reduces network overhead
Excluded URLs	Skip monitoring for analytics/CDN URLs	Reduces beacon volume
Data collection level	`OFF` / `PERFORMANCE` / `USER_BEHAVIOR` per user (MOBL-09 §3); crash reporting is a separate opt-in	Captures only what the user agreed to
Cost and traffic control	Monitored-session percentage in the app settings	Reduces DEM unit consumption

Configuring Excluded URLs

Exclude third-party analytics and CDN URLs that generate noise without providing actionable insight. The filters work on the device, so excluded requests are never captured.

iOS — `DTXURLFilters` in `Info.plist`, an array of URL patterns (wildcards supported):

```
&lt;key&gt;DTXURLFilters&lt;/key&gt;
&lt;array&gt;
    &lt;string&gt;https://analytics.google.com/*&lt;/string&gt;
    &lt;string&gt;https://cdn.myapp.com/*&lt;/string&gt;
    &lt;string&gt;https://firebaselogging.googleapis.com/*&lt;/string&gt;
&lt;/array&gt;
```

Android — the `urlFilters` property of the `webRequests` block in the Dynatrace Android Gradle plugin configuration (OneAgent for Mobile 8.339+). The plugin documentation's *Filter web requests by URL* page has the exact syntax.

> **Correction (10/02/2026).** Earlier revisions showed an Android `dynatrace.config.xml` with `<excludeURLs>` entries. No such file is documented; Android configuration lives in the Gradle plugin DSL.
>
> **Sources:** Configuration — iOS (DT docs) — *"DTXURLFilters Array [] An array of URL patterns to exclude from automatic instrumentation. Supports wildcards."*; What's new in OneAgent for Mobile 8.339 (DT docs) — *"The Dynatrace Android Gradle plugin now supports a new urlFilters property in the webRequests configuration."*

Data Collection Levels

There is no "crash-only" or "user actions only" level. The SDK has three data collection levels -- `OFF`, `PERFORMANCE` and `USER_BEHAVIOR` -- and crash reporting is a separate opt-in (`crashReportingOptedIn`). See MOBL-09 §3 for what each level captures and how to set it.

Cost and Traffic Control

Reducing the share of monitored sessions lowers DEM unit consumption for high-traffic apps. The setting is in the mobile app's settings under **General > Enablement and cost control**, and applies to all users of that application configuration. Sessions that are not monitored also send no business events.

> **Correction (09/28/2026).** Earlier revisions listed a four-level "performance collection level" table (Full / User actions / Crash-only / Off) and placed sampling under "Application Settings > Data Privacy > Session Replay & Data Collection". Neither matches the documentation.

> **Sources:** OneAgent SDK for Android (DT docs) — *"The possible values for the data collection level are as follows: OFF PERFORMANCE USER_BEHAVIOR"*, Session Replay Classic for Android (DT docs) — *"From the application settings, select General > Enablement and cost control ."*

Query Feature Flag Event Tracking

Use this query to see which feature flags are most actively evaluated and which variants are being served:

```
// Feature flag event tracking
fetch bizevents, from:-24h
| filter event.type == "com.myapp.feature_flag"
| summarize usage_count = count(), by:{feature.name, feature.variant}
| sort usage_count desc
| limit 20
```

7. Multi-App Strategies

Many organizations operate multiple mobile applications -- a customer-facing app, a driver/courier app, an internal admin app, or white-label variants for different brands. Managing Dynatrace monitoring across multiple apps requires careful planning.

Shared vs. Separate Configurations

Strategy	Description	Best For
Separate app configs	Each app has its own Dynatrace application ID and configuration	Apps with distinct user bases and SLOs
Shared app config	Multiple app variants share one configuration	White-label apps with identical codebases
Hybrid	Shared config for variants, separate for distinct apps	Organizations with both scenarios

Multi-App Considerations

- **Naming conventions** -- Use consistent prefixes (e.g., `MyBrand Customer`, `MyBrand Driver`, `MyBrand Admin`) so you can filter and group easily in DQL
- **Shared business events** -- If multiple apps send the same `event.type`, include an `app.name` attribute to distinguish the source
- **Cross-app session linking** -- When a user action in one app triggers backend calls that affect another app, use web request tagging (Section 4) to maintain trace continuity
- **DEM unit budgeting** -- Each app consumes DEM units independently. Use cost and traffic control strategically for high-traffic apps while keeping 100% monitoring for critical apps

Compare Session Volume Across Apps

Use this query to compare daily session volumes (distinct `dt.rum.session.id` per `frontend.name`) across all your mobile applications:

```
// Session volume comparison across all mobile apps
fetch user.events, from:-7d
| filter in(dt.rum.agent.type, {"android", "ios"})
| makeTimeseries session_count = countDistinct(dt.rum.session.id), by:
{frontend.name}, interval:24h
```

Series Complete

Congratulations on completing all 12 notebooks in the **MOBL (Mobile Monitoring)** series! You now have a comprehensive understanding of Dynatrace mobile RUM -- from foundational concepts to advanced instrumentation techniques.

Full Series Recap

Notebook	Title	Focus
MOBL-01	Mobile Monitoring Fundamentals	Architecture, platforms, entity types, beacon data flow

Notebook	Title	Focus
MOBL-02	iOS SDK Setup (Swift & SwiftUI)	SPM/CocoaPods integration, Info.plist, DTSwiftInstrumentor
MOBL-03	Android SDK Setup (Kotlin & Jetpack Compose)	Top-level Gradle plugin, variant configurations, Compose
MOBL-04	Cross-Platform Frameworks	Flutter, React Native, Cordova, .NET MAUI (Xamarin end of support)
MOBL-05	User Action Tracking	Auto and custom actions, rage taps, action queries
MOBL-06	Crash Reporting & ANR Detection	Crash capture, symbolication, ANR, crash queries
MOBL-07	Network Request Monitoring	HTTP monitoring, trace correlation, status-code trends
MOBL-08	Session Replay for Mobile	Enablement, masking, capture percentage, crash replay
MOBL-09	Session Properties & Data Privacy	Session properties, user tagging, data collection levels, opt-in, deletion
MOBL-10	DQL for Mobile Analytics	`user.events` / `user.sessions` query library
MOBL-11	Dashboards & Alerting	KPI dashboards, crash-rate detectors, problem workflows
MOBL-12	Advanced Instrumentation & Optimization	Business events, request tagging, A/B testing, SDK tuning

What's Next?

- **Apply what you learned** -- Implement custom business events and manual instrumentation in your production apps
- **Explore related series** -- Review the **SPANS** series for distributed tracing or **WFLOW** series for automated alerting workflows
- **Stay current** -- Dynatrace regularly updates the mobile SDK with new capabilities. Check the Dynatrace release notes for the latest features

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.